CASE STUDY REPORT

Zomato Demand Forecasting & Rider Planning

YASWANTH YADALA

TABLE OF CONTENTS

Chapter 1 – INTRODUCTION 2

Chapter 2 – Understanding the Business Problem.....5

Chapter 3 – Dataset Understanding.....11

Chapter 4 – Demand Forecasting Methodology 19

Chapter 5 – Locality-Level Demand Allocation and Rider Planning.25

Chapter 6 – Decision Support and Operational Risk Assessment 35

Chapter 7 – Application and Dashboard 44

Chapter 8 – Key Development Decisions and Challenges 51

Chapter 9 – Limitations and Future Scope.....54

Chapter 10 – Conclusion 55

Chapter 1 – INTRODUCTION

1.1 Background

Food-delivery platforms operate in an environment where demand can vary significantly across different days and locations. For a platform such as Zomato, understanding the expected order volume in advance is important because delivery capacity has to be aligned with the anticipated demand. If too few riders are available during a high-demand period, delivery operations may be affected by delays and increased workload. On the other hand, maintaining excessive rider availability when demand is low can lead to inefficient utilization of operational resources.

In this case study, I focused on developing a demand forecasting and rider planning system that connects demand prediction with an operational decision. Instead of limiting the analysis to predicting the total number of future orders, I extended the solution to determine where the expected demand is concentrated and how many riders may be required in those localities.

The case study therefore combines demand analysis, forecasting, locality-level demand allocation, rider planning, operational risk assessment, and decision support into a single workflow.

1.2 Business Context

A city-level demand forecast provides an estimate of the total number of orders expected in the upcoming period. However, this information alone is not sufficient for operational planning.

For example, if the system forecasts a certain number of orders for an entire city, the operations team still needs to know:

- Which localities are likely to contribute most of the demand?
- How much demand can be expected from each locality?
- How many active riders should be positioned in those areas?
- How much standby capacity should be maintained?
- Which localities require greater attention?
- How reliable is the recommendation?

I therefore treated the forecasting problem as a decision-support problem rather than only a prediction problem.

The final objective was to connect the forecast with an actionable operational recommendation.

1.3 Problem Statement

The problem I addressed in this case study is:

How can historical order data be used to forecast upcoming demand and translate that forecast into locality-level rider requirements so that delivery capacity can be planned more effectively?

The challenge involves two levels of planning.

At the city level, I need to estimate the expected demand for the upcoming day.

At the locality level, I need to determine how that expected demand may be distributed across different areas and convert the locality-level demand into recommended rider capacity.

An additional challenge is that not every locality necessarily contains enough historical information to support a reliable forecast. Therefore, I also needed to ensure that the system does not generate a seemingly precise operational recommendation when the underlying locality data is insufficient.

1.4 Motivation

I chose this problem because demand forecasting becomes more useful when it directly supports a business decision.

A forecast such as *"53 orders are expected"* provides information, but it does not by itself tell an operations team what action should be taken. I wanted the final system to answer the next set of practical questions:

Where should attention be focused? How many riders should be deployed? How much standby capacity should be maintained? And how confident should I be in the recommendation?

This motivated me to develop the case study as an end-to-end decision-support system rather than as an isolated machine-learning model.

1.5 Case Study Objective

The primary objective of this case study is to develop a system that can:

1. Analyze historical order demand patterns.
2. Forecast upcoming city-level demand.
3. Analyze demand at the locality level.
4. Identify localities with comparatively higher expected demand.
5. Allocate the city-level forecast across operational localities.
6. Convert expected locality demand into recommended rider requirements.
7. Determine standby rider requirements.
8. Classify locality priority based on expected demand.
9. Assess operational risk.
10. Provide a confidence measure for the resulting recommendation.
11. Handle localities where sufficient historical data is unavailable.
12. Present the complete analysis through an interactive dashboard.

1.6 Scope of the Case Study

The scope of my case study extends from historical order data to an operational recommendation.

The workflow covers:

Data → Demand Analysis → Forecast → Locality Allocation → Rider Planning → Risk & Confidence → Decision Support

The system is intended as a planning and analytical support tool. The rider numbers generated by the system should therefore be interpreted as recommendations based on the assumptions and rules defined in this case study, rather than as exact real-world staffing requirements.

This distinction is important because the final recommendation depends not only on the forecasting model but also on the locality demand allocation and rider-planning assumptions introduced during the development of the solution.

Chapter 2 – Understanding the Business Problem

2.1 The Demand Planning Challenge

Food-delivery operations depend heavily on the ability to match delivery capacity with customer demand. The number of orders received by a platform can vary from one day to another, and this variation can also differ considerably across locations within the same city.

I therefore considered demand forecasting as the first stage of the planning problem. The objective is not simply to estimate how many orders may occur, but to use historical demand patterns to provide an estimate that can support the next operational decision.

A city-level forecast provides the expected demand for the city as a whole. However, this information alone is not sufficient for rider planning because delivery operations are distributed across multiple localities. Two days with a similar total number of orders can still create very different operational requirements if the demand is concentrated in different areas.

This led me to treat the problem as a combination of demand forecasting and operational planning.

2.2 The Rider Allocation Challenge

After obtaining the expected demand, the next question is how that demand should be translated into rider requirements.

A simple approach would be to use the total city forecast and directly calculate the number of riders required. However, this would ignore the geographical distribution of orders. Riders are required in specific operating areas, so I needed to determine which localities were likely to experience higher demand.

I therefore introduced a locality-level planning stage between forecasting and rider allocation.

The overall reasoning became:

Expected city demand

↓

Identify demand concentration

↓

Estimate locality-level demand

↓

Calculate rider requirement

↓

Determine standby requirement

This approach allowed me to connect the forecasting output with a practical operational decision.

2.3 The Locality-Level Planning Challenge

The locality-level component introduced an additional data problem.

Some localities contained a substantial amount of historical information, while others had relatively few observations. If I generated a locality-level forecast for every locality regardless of the amount of available data, the system could produce a numerical recommendation that appeared valid even though it was not supported by sufficient historical information.

I therefore decided that locality-level forecasting should be subject to a minimum data requirement.

I set the minimum historical-record requirement to 30 records. A locality with fewer than 30 historical records is treated as having insufficient information for reliable locality-level forecasting.

This decision also affects rider planning. When sufficient historical data is not available, the system does not treat the locality as having zero demand and then interpret that as a valid staffing decision. Instead, it reports that the locality-level forecast is unavailable because sufficient historical records are not available.

This distinction became an important part of the final case study.

2.4 From Forecasting to Decision Support

During the development of the case study, I realized that a forecast by itself would not provide enough information for an operational user.

For example, an output such as:

Expected Orders: 53

only answers the question of how much demand is expected at the city level.

For planning purposes, I also needed to answer:

- Where is the demand concentrated?
- How many riders are recommended?
- How many standby riders should be maintained?
- Which localities require greater attention?
- What is the operational risk?
- How confident is the recommendation?

I therefore extended the system beyond forecasting and developed a decision-support layer around the forecast.

The final decision-support process can be represented as:

Historical Order Data

↓

Demand Analysis

↓

City-Level Forecast

↓

Locality Demand Scoring

↓

Locality Forecast Allocation

↓

Rider Recommendation

↓

Standby Rider Recommendation

↓

Priority Classification

↓

Operational Risk

↓

Decision Confidence

↓

Operational Recommendation

2.5 Key Business Questions

I designed the case study around a set of practical questions that the final system should answer.

City-Level Questions

1. How many orders are expected for the upcoming day?
2. How does the forecast compare with the historical demand level?
3. Is the expected demand low, moderate, or high?
4. How reliable is the forecast?

Locality-Level Questions

1. Which localities are expected to contribute the highest demand?
2. How is the city-level forecast distributed across localities?
3. Does the selected locality have enough historical information to support a forecast?
4. What is the expected demand for the selected locality?

Rider Planning Questions

1. How many riders should be recommended for the expected demand?
2. How many standby riders should be maintained?
3. Which localities require higher rider availability?
4. What operational action should be taken based on the expected demand?

Decision-Support Questions

1. What is the operational risk associated with the expected locality demand?
2. How confident is the system in the recommendation?
3. Why has the system produced the recommendation?
4. What should an operations planner pay attention to?

These questions became the foundation for the calculations and visual components that I later implemented in the application.

2.6 Case Study Approach

Based on the above requirements, I structured the case study into two connected analytical levels.

At the first level, I focused on forecasting overall city demand using historical order information. This provided the total expected demand for the upcoming period.

At the second level, I used locality-level demand information to determine how the city-level demand could be distributed across operational areas. I then converted the resulting locality demand into rider requirements and added operational indicators such as priority, risk, and decision confidence.

This approach allowed me to move from a predictive question:

How much demand should I expect?

to an operational question:

What should I do with that expected demand?

That transition from prediction to action is the central idea behind this case study.

2.7 Overall Business Flow

The complete business flow that I followed is:

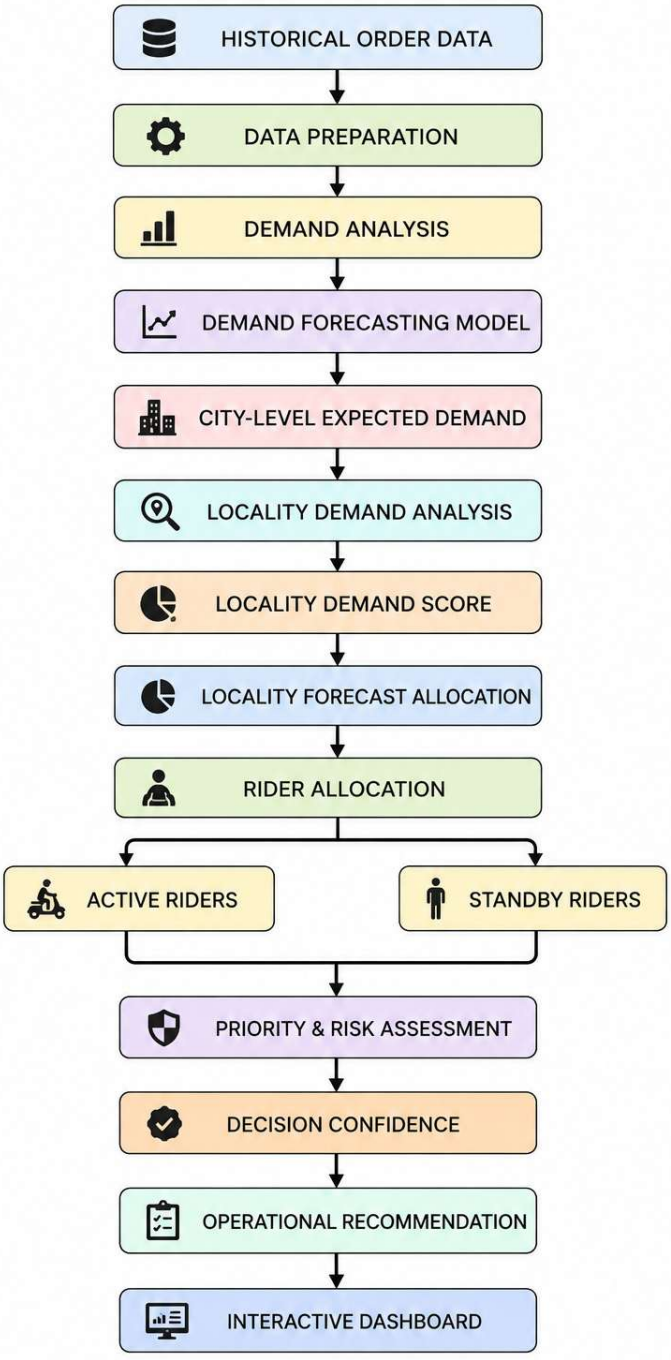


Figure 1: End-to-End Workflow of the Demand Forecasting and Rider Planning Case Study

Chapter 3 – Dataset Understanding

3.1 Data Source

I used a Zomato order dataset as the primary source for this case study. The dataset contains individual order-level information along with details related to the order, customer location, delivery, restaurant, and other operational attributes.

I used the order-level records as the foundation for the demand analysis because they allow the demand pattern to be studied at different levels. In particular, I used the date information to analyze demand over time and the city and locality information to analyze the geographical distribution of orders.

The original dataset was also important for validating the availability of historical records for individual localities. During development, I found that relying only on an aggregated locality-level table could give an incorrect impression of how much historical data was actually available. I therefore used the original order-level data when checking the number of historical records associated with a selected locality.

3.2 Dataset Structure

The dataset contains information across several categories. The important fields I used during the development of the case study include:

Category	Important Fields	Purpose
Order information	order_id, order_date, order_status	Identifying and analyzing orders
Location information	city, locality, state, area	City and locality-level analysis
Delivery information	delivery_time_mins, delivery_distance_km, delivery_agent_id	Operational context
Order characteristics	number_of_items, average_cost_for_two, type	Understanding order characteristics
Customer/payment information	payment_method, tip_amount_inr, has_free_delivery	Additional order context
Restaurant information	name, type, cuisines, rating, aggregate_rating, votes	Restaurant-level context
Geographic information	latitude, longitude	Location-related analysis

I did not use every available column directly in the forecasting and rider-planning calculations. Instead, I focused on the variables that were relevant to demand, time, locality, and operational planning.

This helped me keep the analysis focused on the actual business problem rather than trying to use every available variable simply because it was present in the dataset.

3.3 Data Coverage

The order date was particularly important because the forecasting problem depends on identifying historical demand patterns and generating a forecast for the next available operational day.

During the final application workflow, the system identifies the latest available historical date and generates the forecast for the following date. For example, when the latest available operational data is 01 October 2024, the application presents 02 October 2024 as the forecast date.

This approach ensures that the forecast is positioned as a future operational estimate rather than being presented as a prediction for a date that is already present in the historical data.

3.4 City and Locality Information

I used the **city** and **locality** fields to introduce two levels of geographical analysis.

At the city level, I calculated the overall demand and generated the city-level forecast. This provided the total expected order volume that needed to be planned for.

At the locality level, I analyzed how demand was distributed across different operating areas within the selected city. This was necessary because rider deployment is ultimately a location-specific decision.

For example, a city may have a moderate overall forecast while a few specific localities account for a relatively large portion of the expected demand. Such localities require more operational attention even when the overall city demand does not appear unusually high.

3.5 Historical Demand and Recent Demand

I used historical and recent demand information to represent the demand behavior of each locality.

Historical demand provides a broader view of how a locality has performed over the available observation period. Recent demand, on the other hand, provides an indication of what has been happening more recently.

I considered both because relying only on historical demand can make the system less responsive to recent changes, while relying only on recent demand can make the system too sensitive to short-term fluctuations.

For the locality demand score, I used the following weighting:

Locality Demand Score = 75% × Historical Demand Component

+ 25% × Recent Demand Component

I assigned greater weight to the historical component because I wanted the locality ranking to remain primarily grounded in its established demand pattern while still allowing recent behavior to influence the result.

3.6 Preparation of Locality-Level Data

Before performing locality-level allocation, I prepared an aggregated locality-level dataset containing indicators such as:

- locality
- historical_orders
- recent_orders
- historical_norm
- recent_norm
- demand_score
- orders

These variables were used to compare localities on a common basis.

The historical and recent demand values were normalized before combining them into the demand score. This was necessary because the two components may exist on different scales. Normalization allowed me to combine them into a single score without allowing one component to dominate simply because of its numerical scale.

The resulting demand score was then used to rank localities according to their relative expected demand.

3.7 Data Sufficiency Consideration

One of the important issues I identified during development was that the presence of a locality in the dataset does not automatically mean that enough historical information is available for forecasting.

For example, a locality might appear in the dataset but contain only a small number of historical records. Generating a locality-level forecast in such a situation could result in an operational recommendation that appears precise but is not sufficiently supported by data.

To address this, I introduced a minimum historical-record requirement of 30 records for locality-level forecasting.

The logic is:

Historical records < 30

↓

Insufficient locality data

↓

Locality forecast unavailable

↓

No automatic rider allocation

↓

Operational status = NOT READY

When at least 30 records are available, the locality is considered eligible for locality-level forecasting.

This threshold is an analytical assumption introduced for this case study. It is not a value provided by the original dataset.

3.8 Why the Original Data Was Used for Record Validation

During development, I initially checked locality record availability using an already-processed locality-level dataset. I found that this could produce misleading results because the processed dataset had already undergone aggregation.

For example, the application could report only one record for a locality even though the original order-level dataset contained several individual orders for that locality.

I therefore changed the validation logic so that the number of historical records is calculated directly from the original dataset using the selected city and locality:

Selected City

+

Selected Locality

↓

Filter original order-level data

↓

Count historical records

↓

Compare with 30-record requirement

This change made the data-sufficiency check more representative of the actual historical information available for the selected locality.

3.9 Data Quality and Operational Considerations

I treated data quality as part of the decision-making process rather than as a separate preprocessing step.

The system does not assume that every available locality can always produce a reliable operational recommendation. When sufficient records are unavailable, the system explicitly communicates the limitation instead of converting the lack of information into a zero-demand assumption.

This distinction is important because:

No available forecast ≠ Zero expected demand

A zero forecast could mean that the model expects no demand, whereas an unavailable forecast means that there is not enough information to make a reliable prediction.

I therefore designed the final system to keep these two situations separate.

3.10 Data Preparation Outcome

After preparing the data, I had the information required to perform the next stages of the case study:

Order-Level Data

↓

Date-Based Demand Data

↓

City-Level Demand Data

↓

Locality-Level Demand Data

↓

Historical & Recent Demand Indicators

↓

Locality Demand Score

↓

Forecasting and Allocation

The prepared data therefore became the foundation for both the forecasting component and the operational planning component.

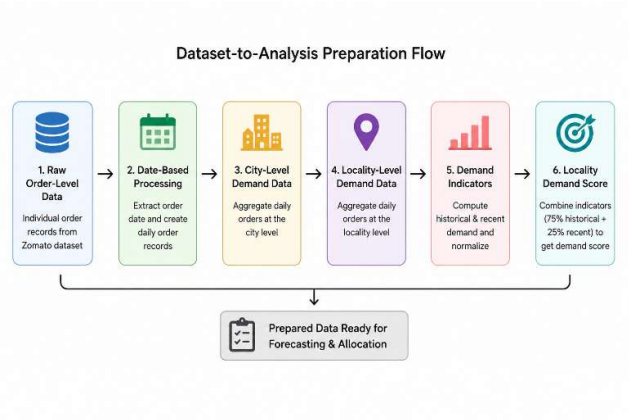


Figure 2: Dataset-to-Analysis Preparation Flow

3.11 Demand Indicator Construction

After preparing the city-level and locality-level demand data, I created additional demand indicators to make the raw order counts more useful for forecasting and operational planning.

I considered both historical demand and recent demand because relying only on the overall historical average could hide recent changes in customer activity. For each locality, I therefore calculated indicators representing its longer-term demand pattern and its more recent demand behaviour.

The historical demand indicator represents the locality's overall demand relative to other localities. The recent demand indicator gives more importance to the latest available observations and helps identify localities where demand has recently increased or decreased.

I then normalized these indicators so that they could be compared across localities without being dominated simply by differences in order volume.

3.12 Locality Demand Score

To convert the demand indicators into a single measure that could be used for rider allocation, I created a locality-level demand score.

I assigned a weight of 75% to historical demand and 25% to recent demand. I selected this weighting because the objective was to maintain stability from established demand patterns while still allowing recent changes to influence the allocation.

The demand score was therefore derived conceptually as:

Demand Score = (75% × Historical Demand) + (25% × Recent Demand)

The resulting score was used to rank the localities according to their expected demand contribution.

This ranking became important later in the rider planning stage because I did not distribute the forecast demand equally across all localities. Instead, I used the demand score to identify the localities that should receive a larger share of the forecast demand.

3.13 Why I Used a Demand Score

I introduced the demand score because the city-level forecast alone was not sufficient for operational decision-making.

For example, if the system predicts a certain number of orders for an entire city, that number does not tell me where those orders are likely to occur. Rider planning requires a locality-level view so that riders can be positioned closer to areas where demand is expected to be higher.

The demand score therefore acts as the connection between the city-level forecast and locality-level rider allocation.

The overall logic I followed was:

City Forecast → Locality Demand Ranking → Locality Forecast Distribution → Rider Requirement → Standby Rider Requirement

This approach allowed me to convert the forecasting output into an operational recommendation rather than stopping at the prediction of order volume.

3.14 Data Prepared for Forecasting and Allocation

At the end of the data preparation stage, I had the following major inputs available:

- Historical daily demand at the city level
- Historical daily demand at the locality level
- Historical and recent demand indicators
- Locality demand scores
- City-level data required for forecasting
- Locality-level data required for forecast distribution
- Historical record counts required for checking data sufficiency

These prepared datasets formed the foundation for the next stage of the case study, where I developed the demand forecasting process and subsequently converted the forecast into rider allocation recommendations.

Chapter 4 – Demand Forecasting Methodology

4.1 Overview of the Forecasting Approach

After completing the data preparation and demand-score construction, I developed the demand forecasting stage to estimate the number of orders expected for the selected city on the forecast date.

The main purpose of this stage was not only to predict future orders but also to produce a forecast that could be used directly for operational planning. The forecast generated at the city level was therefore passed to the locality allocation stage, where I distributed the expected demand across the most relevant localities.

The overall forecasting process followed this sequence:

Historical Demand Data → Forecasting Model → Forecast Orders → Forecast Validation → Demand Status → Locality Allocation

The forecast output was subsequently used for rider requirement calculations.

4.2 Forecast Target

The primary target of the forecasting stage was the number of orders expected for a city on the next forecast date.

I used the historical daily order information available in the dataset to understand the demand pattern before generating the forecast.

For example, if the latest available operational data was from 1 October 2024, the system generated a forecast for 2 October 2024.

This approach allowed the system to operate as a next-day demand forecasting and planning system rather than simply reporting historical demand.

4.3 Historical Average

Along with the forecast value, I calculated the historical average demand.

The historical average provides a baseline against which the forecast can be compared. Instead of interpreting the forecast value in isolation, I used this baseline to understand whether the expected demand was below, close to, or substantially above the normal historical level.

For example, in the interface, the forecast summary can show:

- Expected Orders: 53
- Historical Average: 121.64
- Forecast Reliability: Medium

The historical average therefore acts as a reference point for the decision-support layer.

4.4 Seven-Day Moving Average

I also incorporated the seven-day average as a measure of recent demand behaviour.

The reason for using a seven-day period was to capture the latest demand pattern while reducing the effect of individual daily fluctuations.

A single day can sometimes contain unusually high or low demand. Using a short recent-period average provides a more stable representation of the latest demand conditions.

The seven-day average was therefore retained as one of the important summary indicators presented by the system.

4.5 Forecast Generation

I used the prepared city-level daily demand data as the input to the forecasting component.

The forecasting component generated the expected order count for the selected forecast date. This forecast was then passed to the decision-support engine rather than being treated as the final output of the case study.

This was an important design decision because the objective of my case study was broader than simply predicting orders. I wanted the forecast to answer the operational question:

How many orders are likely to occur, where are they likely to occur, and how many riders should be planned?

Therefore, the forecast value became the starting point for the subsequent allocation and rider-planning stages.

4.6 Forecast Reliability

I included a forecast reliability indicator in the application to communicate how dependable the forecasting result is based on the available data.

The system presents the reliability as a simple category such as Low, Medium, or High rather than displaying a complicated statistical measure to the user.

I used this approach because the intended user of the system is an operational decision-maker who needs an understandable indication of forecast reliability rather than a detailed statistical output.

The reliability information is displayed alongside the expected orders and historical average in the Forecast Summary section of the interface.

4.7 Data Sufficiency as a Forecasting Condition

One of the important assumptions I introduced during development was that a locality-level forecast should not be generated when there is insufficient historical data.

I initially encountered a situation where a locality with very few historical records was still producing zero rider allocation. Although the numerical output appeared valid, it did not clearly communicate that the real problem was insufficient data.

I therefore introduced a minimum-record requirement for locality-level forecasting.

I set the minimum number of historical records required for locality-level forecasting to:

MIN_RECORDS = 30

The system checks the number of historical records available for the selected locality before generating locality-level planning results.

If fewer than 30 records are available, the system does not present the locality as having zero demand. Instead, it explicitly reports that the forecast is unavailable because sufficient historical records are not available.

This distinction was important because:

Zero forecast ≠ Insufficient data

A zero forecast could imply that demand is genuinely negligible, whereas insufficient data means that the system does not have enough evidence to make a reliable locality-level prediction.

4.8 Forecast Date Assumption

I designed the application to generate the forecast for the date immediately following the latest available operational data.

For example:

Latest Available Data: 01 October 2024

Forecast Date: 02 October 2024

This assumption keeps the forecasting process consistent and makes the application suitable for next-day operational planning.

The forecast date is also displayed in the interface so that the user can clearly understand which day the prediction refers to.

4.9 City-Level Demand Classification

After generating the forecast, I classified the expected city-level demand into three categories: Low, Medium, and High.

I derived the classification by comparing the forecast orders with the historical average.

The logic used in the decision-support engine was:

- If forecast orders are below the historical average → Low demand
- If forecast orders are between the historical average and 120% of the historical average → Medium demand
- If forecast orders are at least 120% of the historical average → High demand

This classification was introduced to make the numerical forecast easier to interpret from an operational perspective.

For example, a forecast value by itself may not immediately indicate whether additional operational preparation is required. The demand classification provides a simple interpretation of the forecast relative to the city's normal demand level.

4.10 Forecast Output

The forecasting stage produces several outputs that are subsequently used by the decision-support system.

The main outputs are:

- Forecast orders
- Historical average
- Seven-day average
- Forecast reliability
- Demand status
- Forecast date

These outputs are displayed in the Forecast Summary section of the application.

The forecast orders are then passed to the locality allocation component, which uses the locality demand scores to distribute the predicted city-level demand.

4.11 From Forecasting to Operational Planning

The most important aspect of my forecasting approach is that I did not stop after obtaining the predicted order count.

I connected the forecast directly to the operational planning process.

The complete transition is:

City-Level Forecast

↓

Locality Demand Distribution

↓

Expected Orders per Locality

↓

Recommended Riders

↓

Standby Riders

↓

Operational Risk and Planning Recommendation

This connection transforms the forecasting model into a decision-support system.

Chapter 5 – Locality-Level Demand Allocation and Rider Planning

5.1 Overview of Locality Allocation

After generating the city-level demand forecast, I needed to determine how that forecast would be distributed across individual localities.

A city-level forecast by itself is not sufficient for rider planning. For example, if the system predicts 100 orders for a city, I still need to know which localities are expected to contribute most of those orders.

To solve this, I used the locality demand score developed during the data preparation stage. I ranked the localities according to their demand score and used this ranking to distribute the city-level forecast.

The resulting locality-level forecast was then used to calculate the required number of active riders and standby riders.

The overall process was:

City Forecast → Rank Localities → Select Active Zones → Distribute Forecast → Calculate Riders → Calculate Standby Riders

5.2 Ranking Localities by Demand

I first sorted the locality allocation data using the **demand_score** in descending order.

This means that localities with higher demand scores were placed at the top of the allocation table.

The purpose of this ranking was to identify the areas that were most important from an operational demand perspective.

I did not treat every locality equally because the available data showed differences in demand between localities.

5.3 Selection of Active Operational Zones

After ranking the localities, I limited the main allocation to the top 10 localities.

I implemented this using:

Top operational zones = 10 localities

If fewer than 10 localities were available, I used all the available localities instead.

This was an operational assumption rather than a claim that only 10 localities can have demand. I introduced this limit to keep the rider-planning output focused on the highest-priority operational zones instead of spreading the forecast across a large number of low-demand areas.

Therefore, the allocation logic was:

Rank all localities → Select the top 10 → Allocate forecast demand to these zones

5.4 Allocation of Forecast Demand

I allocated approximately 90% of the city-level forecast demand to the selected active zones.

The active demand was calculated as:

Active Demand = City Forecast × 90%

I then distributed this active demand among the selected localities according to their relative demand scores.

The share received by a locality was based on:

Locality Share = Locality Demand Score / Total Active-Zone Demand Score

The corresponding locality forecast was then calculated from this share.

This ensured that localities with higher demand scores received a larger portion of the predicted demand.

5.5 Rounding Adjustment

Because locality forecasts are ultimately represented as whole numbers of orders, the calculated values can create a small difference between the total locality forecast and the original city-level forecast.

For example, if the city forecast is 53 orders, the individual locality allocations may add up to 52 or 54 after rounding.

I therefore added a rounding correction after the locality-level allocation.

The system calculates the difference between the original city forecast and the total allocated forecast and applies this difference to the first allocation row.

This ensures that the final locality allocation reconciles with the city-level forecast.

This was an important implementation decision because the total expected demand shown at the city level should remain consistent with the locality-level allocation used for planning.

5.6 Example of Locality Allocation

For example, suppose the city-level forecast is:

City Forecast = 100 orders

The system first identifies the highest-ranked operational localities based on their demand scores.

It then calculates approximately:

90% of 100 = 90 orders

These 90 orders are distributed among the selected localities according to their relative demand scores.

If the resulting locality values after rounding do not sum exactly to the original forecast, the system applies the rounding correction.

The final locality-level values are then used for rider planning.

This approach allows me to move from a single city-level forecast to a more useful operational view of where the expected demand is concentrated.

5.7 Final Locality Ranking

After calculating the locality-level forecast, I sorted the allocation again based on **forecast_orders** in descending order.

This produces a final ranking based on the actual expected demand rather than only the original demand score.

As a result, the output directly highlights the localities where the largest number of orders is expected.

This information is also used in the Demand Hotspots by Locality visualization in the application.

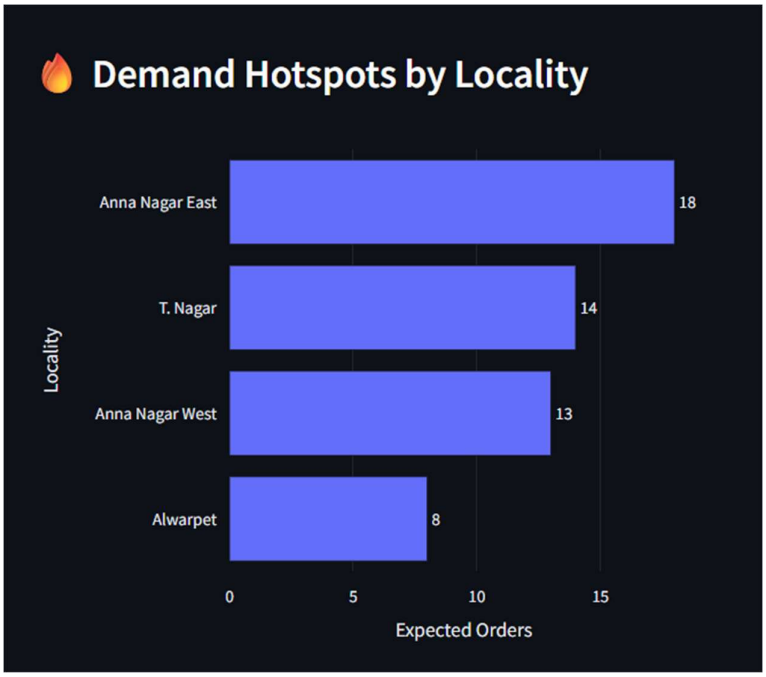


Figure 3: Locality-level forecast demand distribution

5.8 Recommended Rider Calculation

Once the expected orders for a locality were obtained, I converted the forecast into a recommended number of active riders.

I used a dedicated rider recommendation function for this calculation rather than manually assigning riders to individual localities.

The logic takes the locality's expected orders as its input and returns the corresponding recommended rider requirement.

For example, during testing, a locality with an expected demand of 11 orders resulted in a recommendation of 3 riders, while another locality with 14 expected orders resulted in 4 riders.

The important point is that the rider requirement is derived from the forecast demand rather than being manually entered by me.

This creates a consistent relationship between predicted demand and operational staffing.

5.9 Standby Rider Calculation

I also included standby riders to account for possible fluctuations in actual demand.

The standby requirement is calculated using both:

- Expected locality orders
- Recommended active riders

This means the standby requirement is not treated as an independent number.

The purpose of standby riders is to provide additional operational capacity when actual demand exceeds the expected level or when normal rider availability is affected.

For example, the system generated outputs such as:

Recommended Riders: 4

Standby Riders: 2

5.10 Locality Priority Classification

After determining the expected orders and rider requirement, I classified each locality according to its forecast demand using the priority classification logic implemented in the system.

The resulting priority is displayed in the Locality Planning section of the interface.

For example, a selected locality can be displayed with:

Expected Orders: 14

Recommended Riders: 4

Standby Riders: 2

Priority: Critical

This provides an additional operational interpretation beyond the numerical forecast.

5.11 Locality-Level Recommendation

I generated a text-based recommendation for every locality based on its forecast and rider requirement.

The recommendation follows the operational result generated by the system.

When no active rider allocation is required, the system communicates that monitoring should continue.

When riders are required, the system generates a recommendation based on the calculated active and standby rider numbers.

For example:

“Deploy 4 riders and keep 2 standby riders for demand fluctuations.”

This was intentionally designed to make the system useful to someone who may not want to interpret the underlying calculations manually.

5.12 Handling Insufficient Locality Data

I also introduced a separate handling mechanism for localities that do not have enough historical records.

I set the minimum required number of records to:

MIN_RECORDS = 30

If a selected locality has fewer than 30 historical records, I do not treat its demand as zero.

Instead, the system reports that a reliable locality-level forecast cannot be generated because sufficient historical data is not available.

For example, during testing, a locality with only 8 historical records produced a message indicating that the forecast was unavailable because at least 30 records were required.

This distinction was particularly important in the development of the case study because I initially observed that localities with insufficient data could incorrectly appear as having zero expected orders and zero rider requirements.

I corrected this by explicitly checking data sufficiency before proceeding with locality-level forecasting.

5.13 Locality Allocation Output

The final locality allocation table contains the information required for operational planning.

The important fields include:

- Locality
- Historical orders
- Recent orders
- Demand indicators
- Demand score
- Forecast orders
- Recommended riders
- Standby riders
- Priority
- Recommendation

This output forms the main connection between the forecasting component and the rider-planning component.

5.14 Operational Planning Logic

The complete logic I implemented can be summarized as:

1. Generate city-level forecast

↓

2. Calculate locality demand scores

↓

3. Rank localities

↓

4. Select the top operational zones

↓

5. Distribute forecast demand according to demand scores

↓

6. Apply rounding correction

↓

7. Calculate recommended riders

↓

8. Calculate standby riders

↓

9. Assign locality priority

↓

10. Generate operational recommendation

This allowed me to transform the forecasting output into a practical rider deployment plan.

5.15 Key Assumptions in Locality Allocation

The following assumptions were important to the development of this stage:

1. I assumed that localities with higher demand scores should receive a larger share of the city-level forecast.
2. I limited the main operational allocation to the top 10 demand-ranked localities.
3. I allocated approximately 90% of the forecast demand to the active operational zones before applying the final reconciliation step.
4. I used a rounding correction so that locality-level allocation remains consistent with the city-level forecast.
5. I assumed that the number of riders required should be derived from expected orders rather than manually assigned.
6. I included standby riders to provide additional capacity for demand fluctuations.
7. I required at least 30 historical records before treating a locality-level forecast as sufficiently supported.
8. I explicitly distinguished between zero demand and insufficient data. A locality with insufficient records is reported as unavailable rather than automatically being interpreted as having zero demand.

These assumptions are important because they explain how I converted the numerical forecast into the operational recommendations shown in the application.

5.16 Interface Representation

The locality planning results are presented in the application through a dedicated Locality Planning section.

The section displays:

- Expected Orders
- Recommended Riders
- Standby Riders
- Priority
- Deployment Recommendation

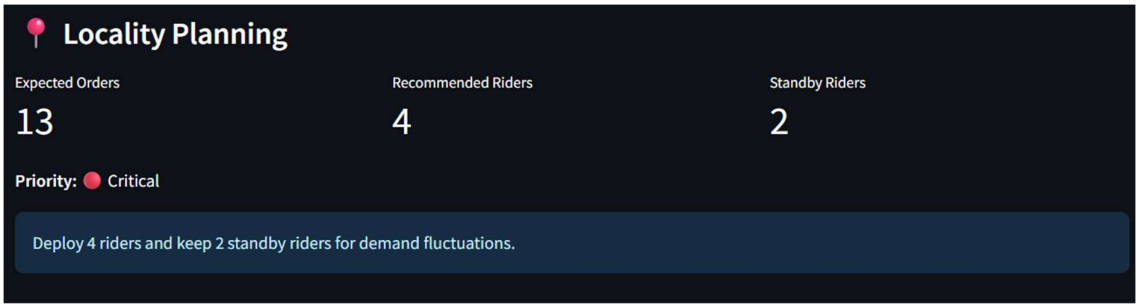


Figure 4: Locality-level rider planning output

The application also presents the highest-demand areas through the Demand Hotspots by Locality visualization.

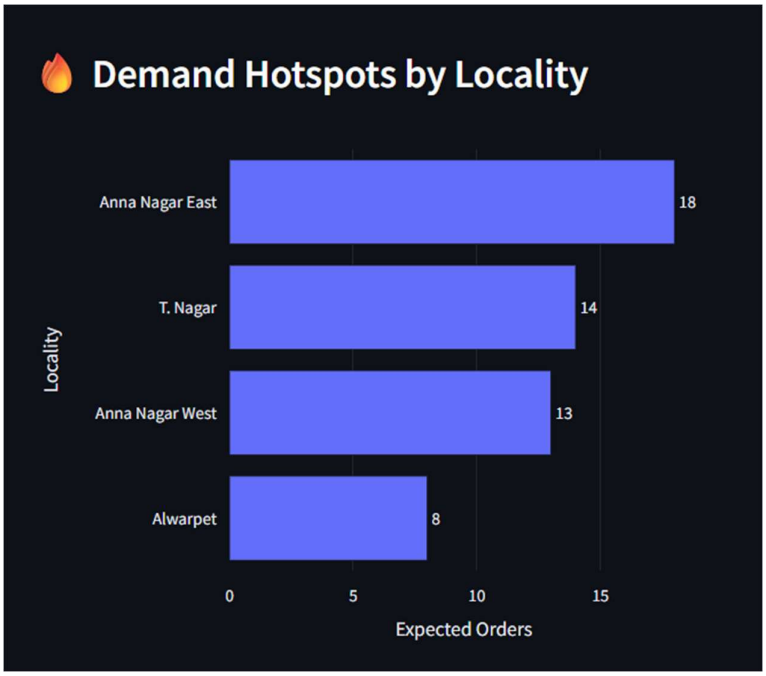


Figure 5: Demand hotspots identified from locality-level forecast

This provides both a numerical and visual representation of the locality-level planning results.

5.17 Outcome of Locality-Level Planning

At the end of this stage, I had converted the city-level demand forecast into an operational locality-level plan.

Instead of simply reporting that a particular city was expected to receive a certain number of orders, the system could now indicate:

- where demand was concentrated,
- how many orders were expected in each priority locality,
- how many riders should be deployed,
- how many standby riders should be maintained, and
- which localities required greater operational attention.

This completed the core demand-to-rider allocation process of my case study.

The next stage focuses on the final decision-support layer, where I combined the forecasting, locality allocation, rider planning, confidence assessment, and operational risk information into a single decision-support output.

Chapter 6 – Decision Support and Operational Risk Assessment

6.1 Overview of the Decision-Support Layer

After completing the forecasting and locality-level rider allocation stages, I developed the final decision-support layer to bring the different outputs together.

The purpose of this layer was to make the forecasting results easier to interpret from an operational perspective. Instead of requiring the user to examine forecast values, locality demand, rider requirements, and other indicators separately, I combined them into a single decision-support view.

The final output provides information about:

- Expected city-level orders
- Historical average demand
- Forecast reliability
- Overall demand level
- Expected locality orders
- Recommended riders
- Standby riders
- Locality priority
- Operational risk
- Locality priorities
- Decision confidence
- Supporting explanations

This made the application more than a forecasting dashboard. It became a system that connects prediction with operational decision-making

6.2 Decision-Support Flow

The final decision-support process follows the complete chain developed throughout the case study:

Historical Data

↓

Data Preparation

↓

Demand Forecast

↓

City-Level Demand Classification

↓

Locality Demand Distribution

↓

Rider Allocation

↓

Standby Rider Planning

↓

Priority and Risk Assessment

↓

Final Operational Recommendation

This is the final analytical flow of my case study.

6.3 City-Level Demand Status

I classified the forecasted city-level demand into three categories:

- LOW
- MEDIUM
- HIGH

The classification is based on the relationship between forecast demand and historical average demand.

I used the following logic:

Forecast < Historical Average → LOW

Historical Average ≤ Forecast < 1.2 × Historical Average → MEDIUM

Forecast ≥ 1.2 × Historical Average → HIGH

This classification allows the user to understand the forecast without having to interpret the numerical values independently.

For example, when the forecast is 53 orders and the historical average is 121.64 orders, the system identifies the city-level demand as LOW.

The interface represents this using a simple visual indicator.

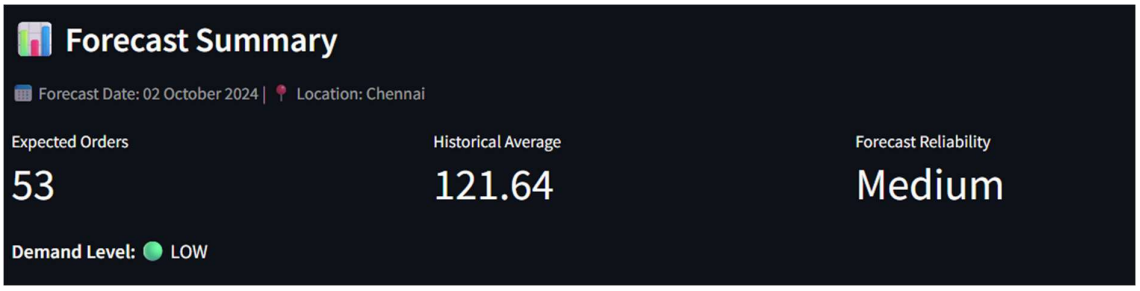


Figure 6: Forecast summary and city-level demand classification

6.4 Operational Risk Assessment

I also included an operational risk indicator to translate the forecast and locality-level demand information into a simple planning signal.

The risk level helps communicate whether additional operational attention may be required.

The system considers the demand situation and locality-level planning results when generating the operational risk information.

For example, the interface can display:

MEDIUM – Locality demand is higher than its usual level. Additional rider availability may be required.

This is more useful operationally than simply displaying a numerical demand value because it directly indicates the possible planning implication.

6.5 Risk-Level Classification

I used three broad risk categories:

- Low
- Medium
- High

The intention was to keep the risk output simple enough for a user to understand immediately.

The risk indicator is not intended to represent a formal statistical probability of failure. Instead, it is a decision-support classification derived from the demand and operational conditions produced by the system.

This distinction is important because the case study is designed primarily as an operational planning tool.

6.6 Handling Insufficient Data in Decision Support

One of the important improvements I made during development was handling insufficient data explicitly.

Initially, when a locality had very few records, the system could return zero forecast orders and zero riders. This could incorrectly make the user believe that the locality had no demand.

I corrected this behaviour by introducing the minimum-record check.

I used:

Minimum required locality records = 30

When the selected locality does not meet this requirement, the decision-support engine returns an insufficient-data status instead of treating the locality as a normal zero-demand locality.

The application can then communicate that the forecast is unavailable because sufficient historical records are not available.

This ensures that:

Insufficient Data ≠ Zero Demand

This was one of the important reliability improvements I made to the case study.

6.7 Decision Confidence

I included a Decision Confidence section to communicate how strongly the available information supports the operational recommendation.

The confidence value is presented as a percentage in the interface.

For example, the application can display:

Decision Confidence: 75%

This provides the user with an immediate indication of the strength of the operational recommendation.

The confidence section is supported by explanatory reasons rather than presenting the percentage alone.

This makes the output more transparent because the user can understand why the system has assigned a particular level of confidence.

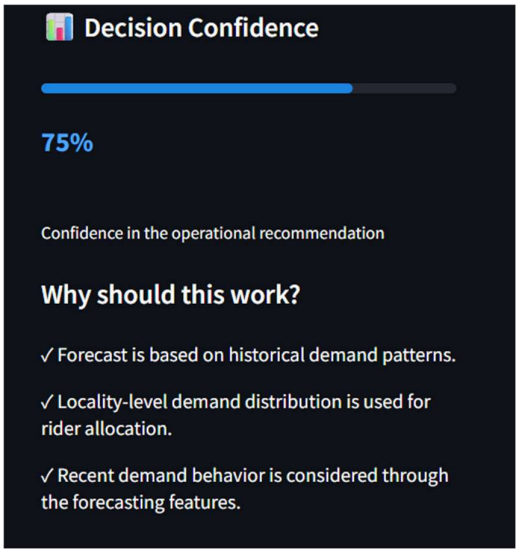


Figure 7: Decision confidence and supporting explanation

6.8 Confidence Factors

The decision confidence is influenced by the information available to the decision-support system.

The main factors considered include:

- Availability of historical demand data
- Availability of locality-level information
- Reliability of the forecast
- Quality of the locality-level demand information
- Ability to generate a locality-level rider recommendation

When sufficient information is available, the system can provide a normal operational recommendation.

When important locality-level information is missing, the system reduces confidence or marks the recommendation as unavailable rather than presenting a misleading result.

6.9 Locality Priority Insights

I also created a locality-priority section to identify areas that require greater operational attention.

The system uses the locality-level forecast and demand ranking to identify the most important demand concentration areas.

The interface displays these localities under the Locality Priority section.

For example, during testing, the system identified localities such as:

- Anna Nagar East
- T. Nagar
- Anna Nagar West

The exact localities displayed change according to the selected city and the underlying forecast results.

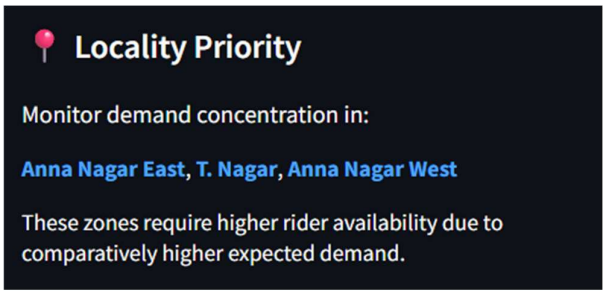


Figure 7: Locality priority recommendations

This provides an operational answer to the question of where rider availability should receive greater attention.

6.10 Rider Allocation Insight

The Rider Allocation section summarizes the rider requirement generated from the locality-level forecast.

The system communicates both active and standby requirements.

For example, the application may display:

Recommended deployment: 4 riders with 2 standby rider(s) to handle demand fluctuations.

This provides a direct operational recommendation without requiring the user to manually calculate rider requirements from expected orders.

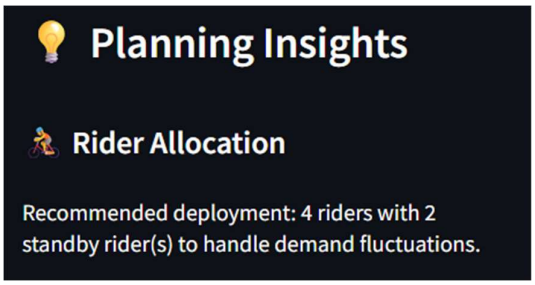


Figure 8: Rider allocation decision support

6.11 Operational Explanation

I intentionally included explanatory text along with the numerical outputs.

For example, instead of displaying only:

4 Riders + 2 Standby

the system communicates:

Deploy 4 riders and keep 2 standby riders for demand fluctuations.

This makes the application more human-readable and closer to how an actual decision-support system would communicate with an operations team.

The user therefore receives both the calculation and its operational interpretation.

6.12 Forecast Trend Visualization

To support the numerical forecast, I included a Forecast Trend visualization.

The chart shows the historical demand pattern over time along with the model forecast.

This allows the user to visually compare the predicted demand with previous observations.

The visualization is particularly useful because a forecast value can be difficult to interpret without knowing the historical behaviour surrounding it.

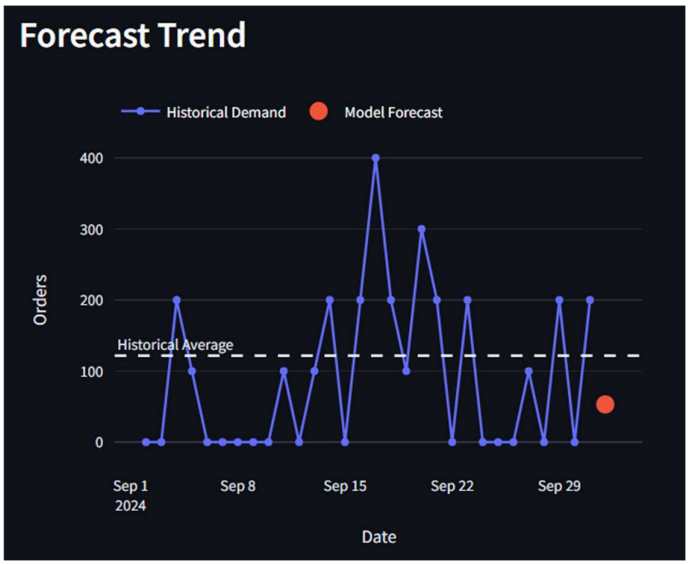


Figure 9: Historical demand and model forecast trend

6.13 Handling Exceptional Conditions

During development, I encountered several situations where the system could produce technically valid but misleading outputs.

I therefore added explicit handling for exceptional conditions.

The main cases addressed were:

Insufficient locality records

If fewer than 30 historical records are available, the locality-level forecast is marked as unavailable.

Locality not found

If the selected locality cannot be matched with the available data, the system does not attempt to generate a normal locality-level recommendation.

Zero or negligible forecast

If the forecast does not require active rider allocation, the system communicates that additional rider deployment is not required instead of generating an unnecessary recommendation.

Missing decision-support information

I also ensured that the decision-support output contains the required insight fields so that the interface does not fail when displaying the final recommendations.

These checks improved the stability of the application during testing.

6.16 Important Development Assumptions

Several assumptions played an important role in the final decision-support design.

I assumed that historical demand provides the main basis for understanding future demand.

I gave recent demand a smaller but meaningful influence through the 75% historical and 25% recent weighting used in the locality demand score.

I assumed that the highest-ranked localities based on demand score should receive priority during forecast allocation.

I limited the main operational allocation to the top 10 localities.

I allocated approximately 90% of the city forecast to these active zones and used a rounding correction to maintain consistency with the city-level forecast.

I assumed that rider requirements should be derived from expected locality demand.

I included standby riders to provide additional capacity for demand fluctuations.

I required at least 30 historical locality records before considering the locality-level forecast sufficiently supported.

I treated insufficient data separately from zero demand to avoid misleading operational recommendations.

These assumptions are not hidden inside the system. They form an important part of how I designed the decision-support logic.

6.17 Final Decision-Support Output

After completing all the stages, the application produces a consolidated operational view.

The user can see the city-level forecast, locality-level demand, rider requirement, standby requirement, operational risk, locality priorities, and decision confidence in the same interface.

This is the final outcome I wanted from the case study: not just a prediction, but a structured recommendation that can support operational planning.

Chapter 7 – Application and Dashboard

7.1 Application Overview

After completing the forecasting and decision-support components, I developed an interactive Streamlit application to bring the complete analysis into a single interface.

The purpose of the application is to allow a user to select a city and locality and then view the corresponding forecast and operational recommendations without having to manually interpret the underlying calculations.

The interface is organized into three main areas:

1. Location selection and forecast generation
2. Forecast and demand analysis
3. Planning insights and operational recommendations

I kept the interface focused on the decisions produced by the case study rather than displaying unnecessary technical information.

7.2 Location Selection

The first part of the application allows the user to select the required city and locality.

After selecting the location, the user can generate the forecast for the next operational day.

This provides a simple starting point for the analysis because all subsequent outputs are based on the selected location.

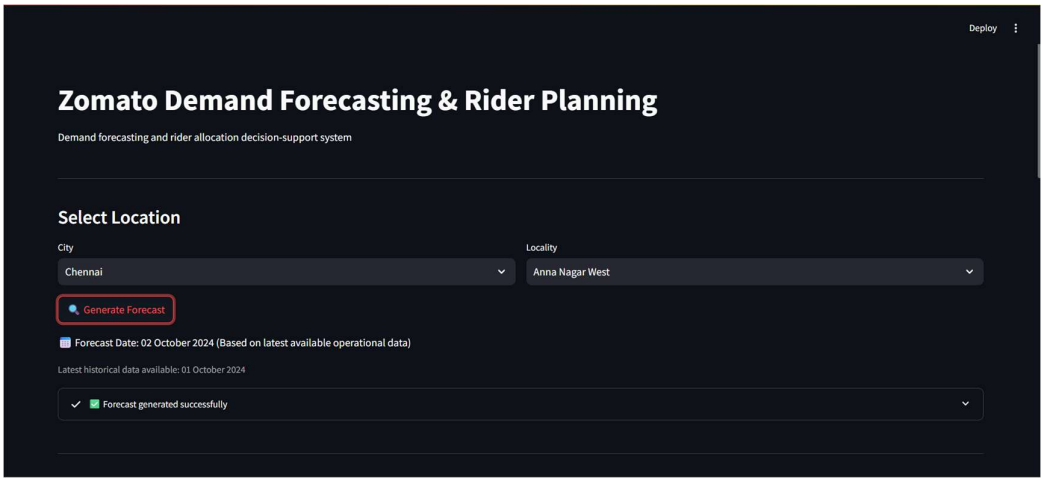


Figure 10: City and locality selection interface

7.3 Forecast Summary

After generating the forecast, the application displays a Forecast Summary containing the main city-level indicators.

The section presents:

- Expected Orders
- Historical Average
- Forecast Reliability

The demand level is also displayed separately so that the user can quickly understand the forecast relative to the historical demand pattern.

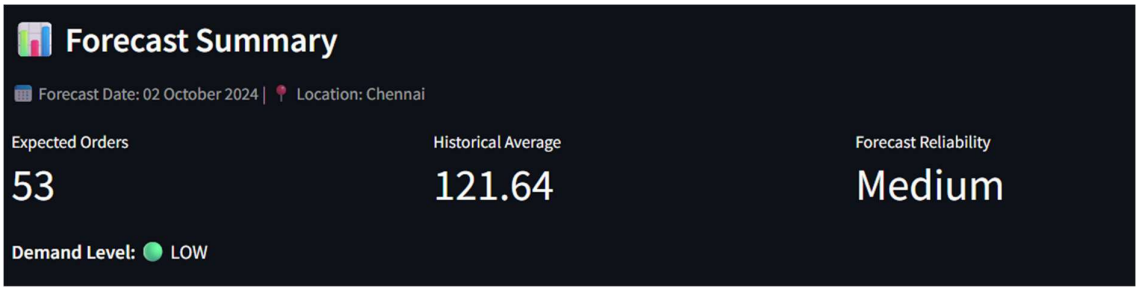


Figure 11: Forecast summary displayed by the application

7.4 Locality Planning

The Locality Planning section provides the main operational output for the selected locality.

It displays:

- Expected Orders
- Recommended Riders
- Standby Riders
- Priority
- Operational Recommendation

This section connects the demand forecast directly with rider planning.

For example, if the selected locality has an expected demand of 14 orders, the application can display the corresponding rider requirement and standby coverage along with the locality priority.

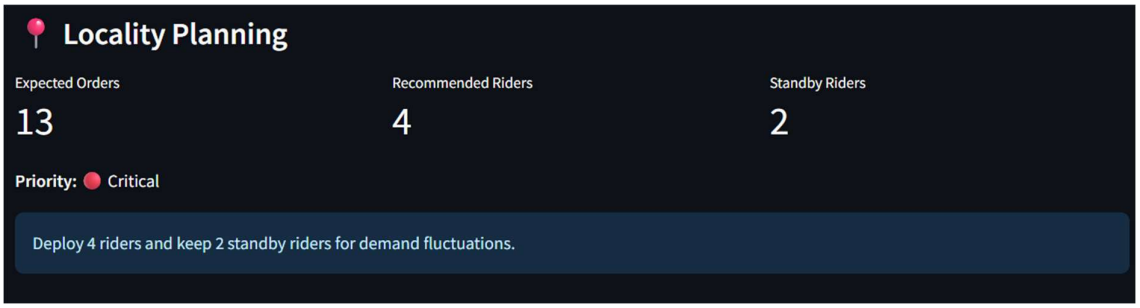


Figure 12: Locality-level rider planning output

7.5 Demand Analysis

The Demand Analysis section contains two visualizations that provide a quick understanding of demand behaviour.

Forecast Trend

The Forecast Trend visualization shows the historical demand pattern together with the model forecast. This allows the user to compare the expected demand with recent historical behaviour.

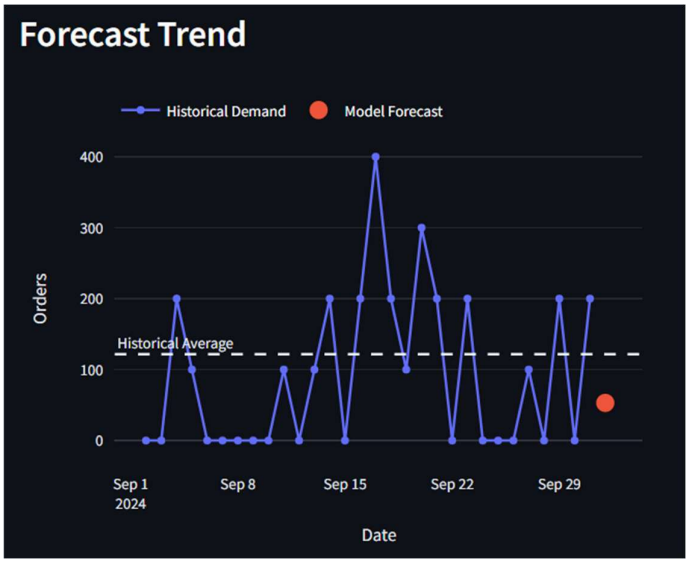


Figure 13: Forecast trend visualization

Demand Hotspots by Locality

The Demand Hotspots visualization displays the localities with the highest expected demand.

This makes it easier to identify areas that may require greater operational attention.

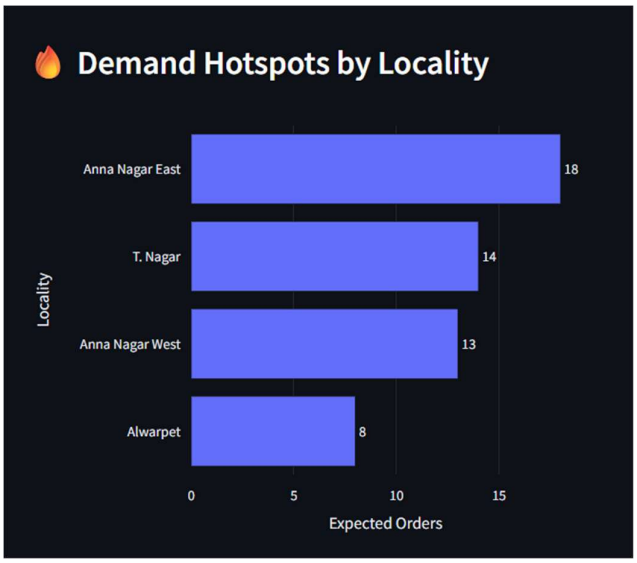


Figure 14: Locality demand hotspots

7.6 Planning Insights

The Planning Insights panel summarizes the important operational outputs generated by the system.

It includes:

- Rider Allocation
- Operational Risk
- Locality Priority
- Decision Confidence

I placed these insights together so that a user can review the key planning information without moving away from the main forecast results.

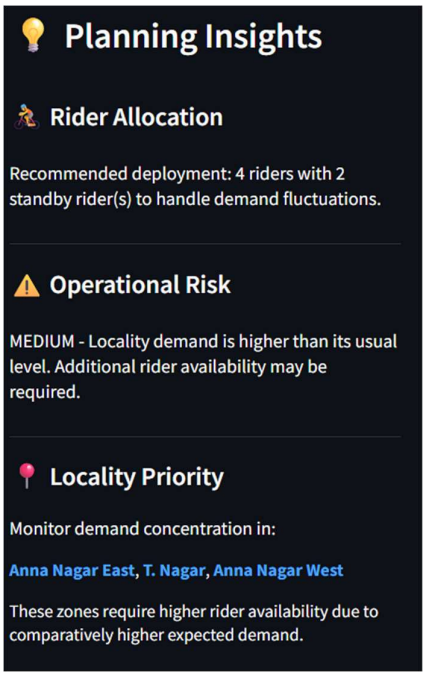


Figure 15: Planning insights panel

7.7 End-to-End User Workflow

The final application can be used through a simple sequence:

Select City and Locality

↓

Generate Forecast

↓

Review Expected Demand

↓

Check Locality Planning

↓

Review Demand Trend and Hotspots

↓

Review Rider Allocation, Risk and Confidence

↓

Use the Recommendation for Operational Planning

This workflow allows the application to move from a raw forecasting result to an interpretable operational decision within a single interface.

7.8 Handling Unavailable Forecasts

The application also provides a specific response when sufficient locality-level historical data is not available.

Instead of displaying a normal rider recommendation, it communicates that the locality-level forecast is unavailable and explains the reason.

This prevents the user from interpreting a lack of data as an indication of zero demand.

This functionality is particularly important because the application is intended to support decisions rather than simply display calculated numbers.

7.9 Application Outcome

The final application brings together the different components developed throughout the case study.

It provides a single interface through which I can move from location selection to demand forecasting, locality analysis, rider planning, and operational decision support.

The application therefore acts as the final presentation layer of the analytical workflow developed in this case study.

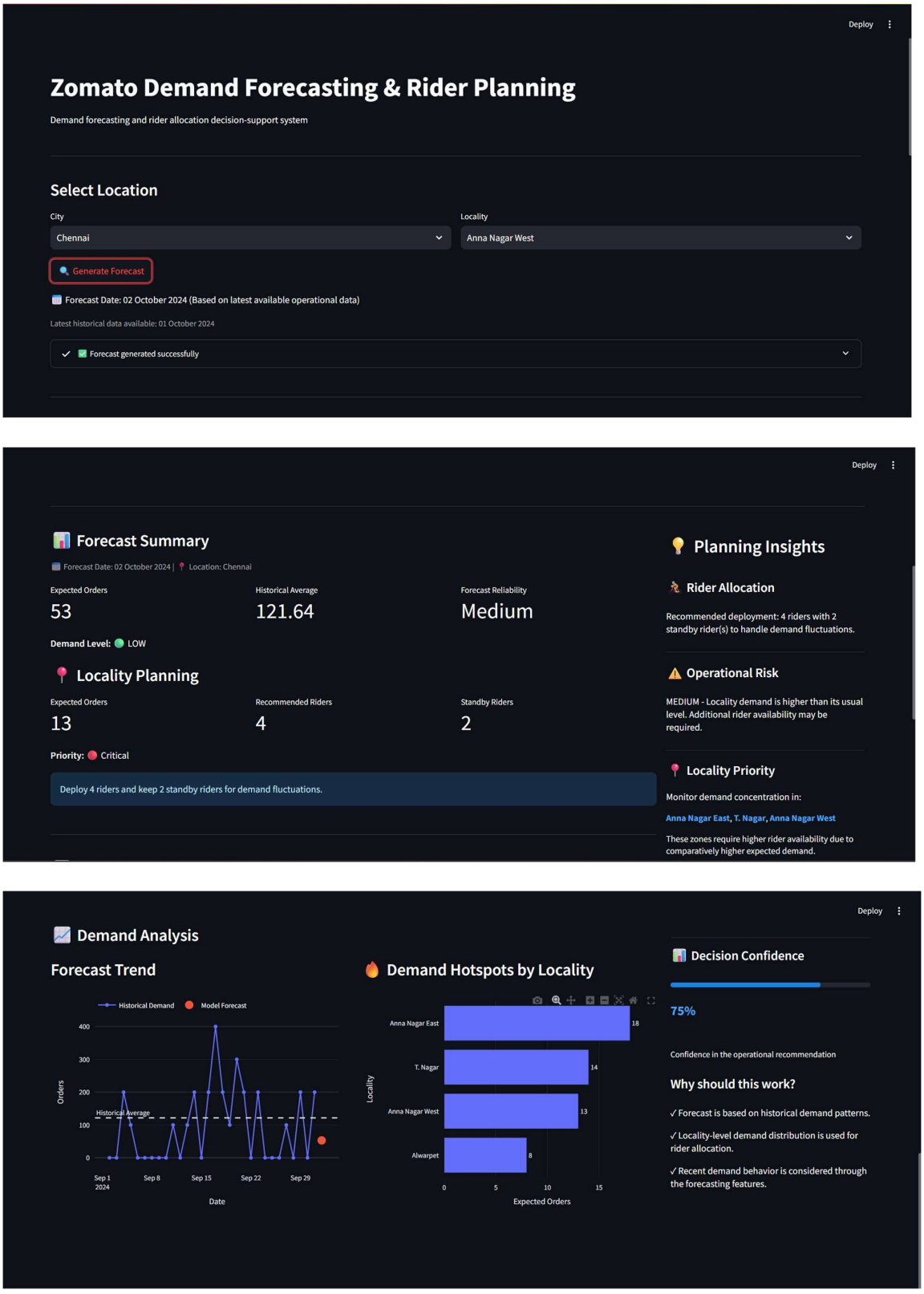


Figure 16: Final demand forecasting and rider planning dashboard

Chapter 8 – Key Development Decisions and Challenges

During the development of the case study, I made several changes after observing how the initial implementation behaved. These changes were important because some early outputs were technically valid but did not represent a useful operational decision.

I therefore refined the system based on the relationship between data availability, forecast demand, locality allocation, and rider planning.

8.1 Locality-Level Allocation

The initial city-level forecast could indicate the expected demand for the entire city, but this was not sufficient for rider planning.

I therefore introduced locality-level demand scoring and used it to distribute the city forecast among the highest-priority operational zones.

This allowed the system to move from a city-wide prediction to a locality-specific planning recommendation.

8.2 Minimum Historical Records

One of the important decisions I introduced was the minimum historical-record requirement for locality-level forecasting.

I set the threshold at 30 historical records.

This was introduced because a locality with very few observations should not automatically receive a forecast and rider recommendation simply because it exists in the dataset.

The final system therefore checks the original order-level data before generating locality-level planning results.

8.3 Distinguishing Insufficient Data from Zero Demand

During testing, I identified an important issue where a locality with no usable forecast could appear as having zero expected orders and zero riders.

I recognized that this could be misleading.

A locality with zero forecast demand and a locality for which a forecast cannot be generated are two different situations.

I therefore changed the system so that insufficient data produces an explicit message explaining that the forecast is unavailable because sufficient historical records are not available.

This became an important reliability feature of the final application.

8.4 Rider Allocation Rule

I initially used a fixed rider-allocation rule based on different order ranges. During testing, I found that this could recommend an unnecessarily large number of riders for low forecast values.

I therefore refined the rider planning logic so that the recommended rider count is more directly related to expected demand.

I also introduced standby riders separately so that additional capacity could be maintained without treating standby riders as part of the normal active deployment.

This separation made the operational recommendation more realistic and easier to interpret.

8.5 Locality-Based Operational Risk

Another issue appeared in the operational risk calculation.

Initially, the risk explanation could compare the selected locality's situation with the overall city-level demand. This could produce an inappropriate risk message when the locality itself had very low or zero expected demand.

I corrected the logic so that locality-level operational risk is based on the demand conditions of the selected locality.

This made the risk output more consistent with the actual decision being made at the locality level.

8.6 Handling Locality Matching

I also added safety checks for situations where the selected locality could not be matched with the available allocation data.

Instead of allowing the application to fail or generate an unsupported recommendation, the system returns an unavailable status and explains that a reliable locality-level allocation cannot be generated.

This improved the robustness of the decision-support workflow.

8.7 Development Outcome

These changes helped me move from an initial forecasting prototype toward a more reliable operational planning system.

The important development principle I followed was that the system should not simply produce a number. It should also communicate whether that number is supported by sufficient data and what operational action can reasonably be taken from it.

This resulted in the final workflow:

Forecast → Validate Data → Allocate Locality Demand → Plan Riders → Assess Risk → Provide Recommendation

Chapter 9 – Limitations and Future Scope

9.1 Limitations

Although the final system provides an end-to-end demand planning workflow, there are several limitations that I identified during the case study.

The first limitation is that the forecasting quality depends on the historical information available in the dataset. Localities with limited historical records cannot support the same level of confidence as localities with sufficient observations.

The second limitation is that the locality allocation is based on demand scores and predefined allocation assumptions. The resulting locality forecast should therefore be treated as an operational estimate rather than an exact prediction of the number of orders that will occur in each locality.

The third limitation is that rider requirements are generated using predefined planning rules. Actual rider requirements in a real delivery operation may also depend on factors such as rider availability, delivery distance, working hours, traffic conditions, order batching, and service-level requirements.

The fourth limitation is that the current analysis operates primarily at the daily demand level. It does not provide detailed hourly or peak-period forecasting.

Finally, the application is intended as a decision-support system. The recommendations should support operational planning rather than completely replace human judgement.

9.2 Future Scope

I identified several areas through which the system could be extended in the future.

The forecasting component could be improved by incorporating additional demand drivers such as holidays, weekends, weather, promotions, restaurant availability, and local events.

The system could also be extended from daily forecasting to hourly demand forecasting. This would allow rider requirements to be planned for specific peak periods rather than only for the complete day.

The locality allocation could be enhanced by incorporating additional geographical and operational factors such as delivery distance, rider availability, traffic conditions, and locality-specific service patterns.

The rider planning component could also be developed into an optimization model that considers both rider availability and operational constraints simultaneously.

With more historical data, the system could additionally support continuous model evaluation and periodic retraining so that the forecasting process adapts to changes in customer ordering behaviour.

Chapter 10 – Conclusion

In this case study, I developed an end-to-end demand forecasting and rider planning system using historical Zomato order data.

I began by preparing the order-level data and analyzing demand at both city and locality levels. I then developed the forecasting stage to estimate upcoming city-level demand and used locality demand indicators to distribute that forecast across the most relevant operational areas.

I extended the forecasting output into a rider planning process by calculating recommended active riders, standby riders, locality priority, and operational risk. I also introduced data-sufficiency checks so that the system does not generate unsupported locality-level recommendations when historical information is inadequate.

The final Streamlit application brings these components together into an interactive decision-support dashboard. A user can select a location, generate the forecast, examine demand trends and hotspots, review locality-level rider requirements, and understand the associated operational insights.

The main outcome of my case study is therefore not simply a predicted order count. I developed a workflow that connects demand prediction with locality-level resource planning and communicates the resulting recommendation in a form that can support operational decision-making.

The project also helped me understand that a useful analytical solution must consider not only model output but also data quality, business assumptions, interpretability, and the practical decisions that follow from the analysis.